

Hash Tables

DSA27 — Lecture 13 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 13 · Fall 2026

Where We Are

Finding a value by its key

Structure	Find a key	Needs
unsorted array, linked list	$O(n)$	nothing
sorted array (Week 8)	$O(\log n)$	sorted order
binary search tree (Week 11)	$O(\log n)$ if balanced	order, balance
hash table (today)	$O(1)$ on average	a good hash function

Today's question: **can we find a key without comparing it to other keys at all?**

1. The dream: the key **is** the index
2. Hash functions: what `hash()` does, and what is and is not reproducible
3. Collisions, and the birthday paradox
4. Chaining: a linked list in every bucket
5. Open addressing: linear probing, clustering, **tombstones**
6. Load factor and resizing — amortised again
7. Measured: probes against load factor; and the keys that break it
8. Python's `dict` and `set`: the real thing, which you now understand

The Dream

Direct addressing: the key is the index

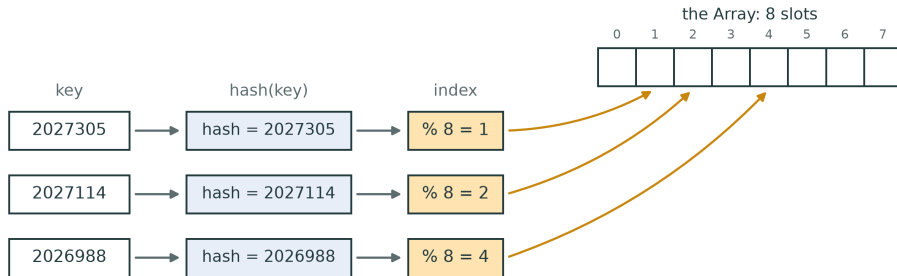
Keys are student numbers 0 to 999? Use an Array (1000):

- $\text{put}(k, v): a[k] = v - O(1)$
- $\text{get}(k): a[k] - O(1)$

No search at all. **But:** the space is the size of the **key range**, not the number of keys.

Hashing: squeeze the key into the table

Hashing turns any key into an index: $\text{hash}(\text{key}) \% \text{capacity}$



$$\text{index} = \text{hash}(\text{key}) \bmod \text{capacity}$$

Hash Functions

Python's hash ()

Expression	Value	Every run?
hash(42)	42	yes
hash(-1)	-2	yes
hash(1.0), hash(True)	1	yes
hash((1, 2))	-3550055125485641917	yes
hash("abc")	a different number each run	no
hash([1, 2])	TypeError: unhashable type: 'list'	—

Same run, same hash. Next run — maybe not.

```
python -c "print(hash('abc'))"      # -60919065288970757
python -c "print(hash('abc'))"      # 2138548942786055565
python -c "print(hash(12345))"      # 12345, every time
```

- **Deterministic within one run:** always — a table depends on it.
- **Across runs:** int, float, tuples of them — yes. **str and bytes — no:** randomised on purpose.
- PYTHONHASHSEED=0 fixes the seed, for reproducing a bug.

What makes a good hash function

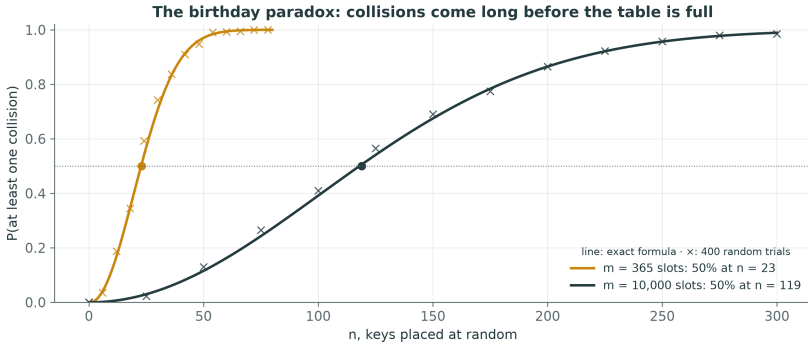
1. **Consistent:** $a == b \Rightarrow \text{hash}(a) == \text{hash}(b)$. Non-negotiable.
2. **Spreads keys evenly** over the indices — similar keys, different slots.
3. **Uses the whole key**, not just part of it.
4. **Fast** — it runs on every single operation.

A bad one: **sum of the character codes**. "listen", "silent" and "enlist" all hash to 655.

Collisions

Collisions are certain

- **Pigeonhole:** more possible keys than slots $\Rightarrow$ some keys **must** share.
- **Birthday paradox:** it happens **early**.



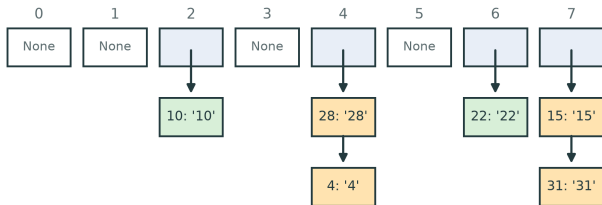
Two ways to live with collisions

	Chaining	Open addressing
a slot holds	a list of entries	one entry
on collision	add to the list	probe another slot
load factor	may exceed 1	must stay < 1
delete	unlink a node	leave a tombstone
used by	Java HashMap	Python dict, set

Chaining

A linked list in every bucket

Chaining: each bucket holds a linked list of Entry nodes



put 10, 22, 31, 4, 15, 28
into 8 buckets:

15 and 31 share bucket 7
28 and 4 share bucket 4

a new key goes on the
FRONT of its chain

`self._buckets` is an Array; each slot is None or the **first** Entry of a chain.

The three operations, in words

get(key): walk the chain at `_index(key)`; return the value of the entry whose `key == key`; at the end of the chain: `default`, or **KeyError**.

put(key, value): walk the same chain; if the key is there, **overwrite** and return. Otherwise push a new Entry on the front; `size += 1`; if `size / capacity > max_load`, **resize** to twice the capacity.

delete(key): walk with **two** references, `prev` and `entry`; unlink the match — the head of the chain is the special case; `size -= 1`. Not found: **KeyError**.

Trace: chaining, capacity 8, max_load 0.75

Operation	Index	Buckets (non-empty)	size	load
put 10, 22, 31, 4	2, 6, 7, 4	2:10 · 4:4 · 6:22 · 7:31	4	0.5
put 15	7	... 7: 15 → 31	5	0.625
put 28	4	4: 28 → 4 ...	6	0.75
put 7	7	7 > 0.75·8: resize to 16	7	0.4375
		4:4 · 6:22 · 7:7 · 10:10 · 12:28 · 15: 31 → 15		
delete 31	15	15: 15	6	0.375

Load factor: $\alpha = n/\text{capacity}$ — the average chain length.

- successful get: about $1 + \alpha/2$ entries examined
- unsuccessful get: about α entries — the whole chain
- keep $\alpha \leq 0.75 \Rightarrow O(1)$ **on average**

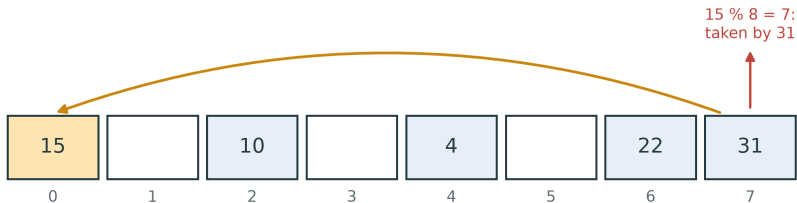
Worst case: every key in **one** chain — $O(n)$.

Open Addressing

Linear probing: one key per slot

Open addressing, linear probing: one key per slot, step on when taken

probe 7, then $(7 + 1) \% 8 = 0$: free — 15 goes there



Two Arrays, `_keys` and `_values`. Start at $\text{hash}(\text{key}) \% \text{capacity}$; if that slot is taken by another key, try the **next**: $(i + 1) \% \text{capacity}$.

Trace: open addressing, capacity 8, max_load 0.66

Operation	Home	Probes	Slots 0-7	size
put 10, 22, 31, 4	2, 6, 7, 4	1 each	_ _ 10 _ 4 _ 22 31	4
put 15	7	7, 0	15 _ 10 _ 4 _ 22 31	5
delete 31	7	7	15 _ 10 _ 4 _ 22 †	4
get 15	7	7 (†, go on), 0	unchanged	4
put 23	7	<i>load check: rebuild first</i>	23 _ 10 _ 4 _ 22 15	5

† is the tombstone.

Primary clustering

Primary clustering: occupied slots clump into runs

a run of 9



40 random keys in 64 slots (load 0.625): 11 runs; the longest are 9, 8, 7, 4. A key that hashes anywhere into a run lands at its end — and makes it longer.

A key that hashes **anywhere** into a run lands at its **end** — and the run grows. Long runs get longer faster.

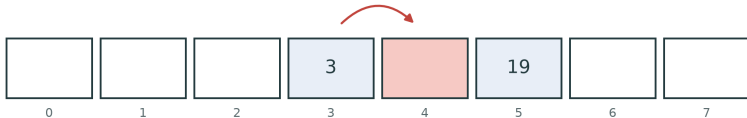
Delete: why not just empty the slot?

Why delete cannot just empty a slot

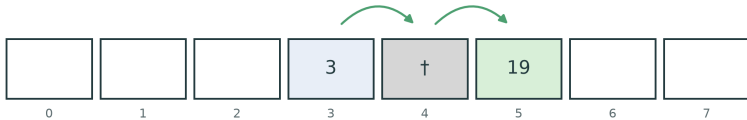
3, 11 and 19 all hash to 3: they sit in slots 3, 4, 5



delete 11 by EMPTYING slot 4 — then get(19) probes 3, 4: None, 'not here'. Wrong.



delete 11 with a TOMBSTONE — get(19) probes 3, 4 (skip), 5: found.



Load Factor and Resizing

Resize: every key moves

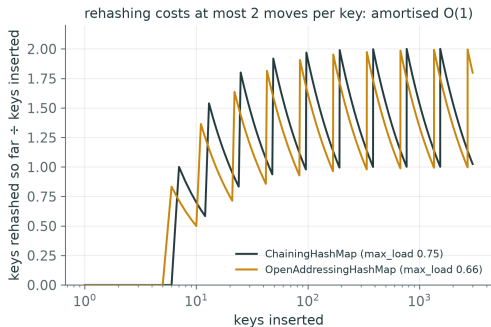
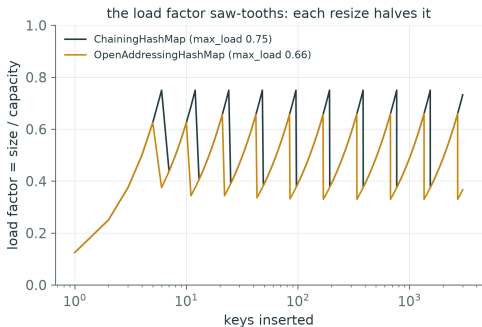
$$\alpha = \frac{\text{size}}{\text{capacity}} \quad \text{chaining: } \alpha \leq 0.75 \quad \text{open addressing: } \alpha \leq 0.66$$

When a put would pass the limit: allocate **twice** the buckets and **re-insert every key**.

You **cannot copy the buckets across**: the index is $\text{hash} \% \text{capacity}$, and the capacity just changed.

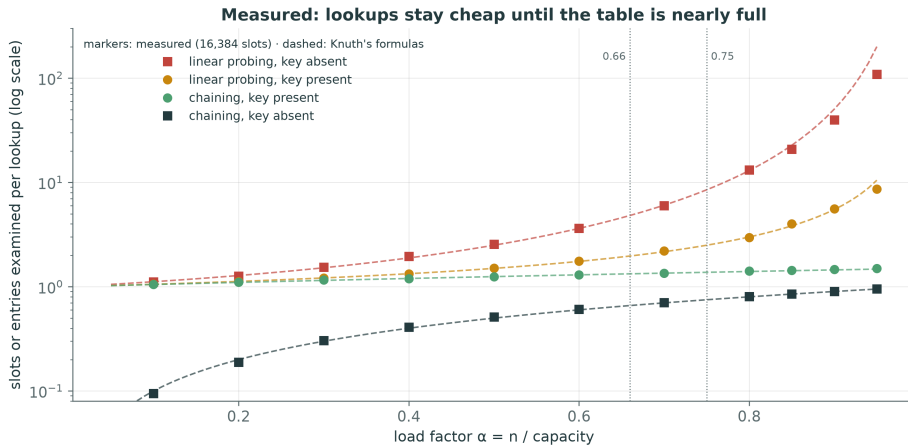
Amortised $O(1)$ — Week 4 again

Measured: doubling keeps the table short, and pays for itself

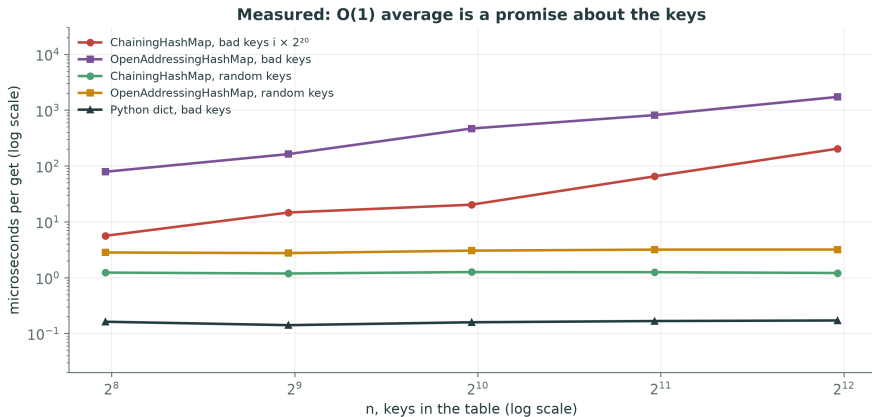


Measured

Probes against load factor



What breaks $O(1)$



Keys $i \times 2^{20}$ all end in twenty zero bits: in **every** table of $2^k \leq 2^{20}$ slots they share **one** bucket. Each get becomes $O(n)$.

The Real Thing

Python's dict and set — which you now understand

- **Open addressing**, capacity a power of two, **8** slots to start.
- Grows at about $\frac{2}{3}$ full (your `max_load` = 0.66).
- A deleted key leaves a **dummy** — a tombstone.
- Probes $i = (5 \cdot i + 1 + \text{perturb}) \bmod 2^k$, with `perturb` feeding in the high bits of the hash.
- Since Python 3.7, a dict keeps **insertion order**. A set does not.

Hash table, sorted array or tree?

	Hash table	Sorted array	Balanced BST
find by key	$O(1)$ avg	$O(\log n)$	$O(\log n)$
insert, delete	$O(1)$ avg, amortised	$O(n)$	$O(\log n)$
min, max, in order range $[a, b)$, next larger	$O(n)$ — no order no	$O(1), O(n)$ $O(\log n)$	$O(\log n), O(n)$ $O(\log n)$
worst case	$O(n)$	guaranteed	guaranteed

This Week

Exercises: dsa/hashmap.py

Class	Method	The trap
Chaining	get	== not is; KeyError only without a default
Chaining	put	overwrite, never duplicate; resize above 0.75
Chaining	delete	the head of a chain; size -= 1
Chaining	_resize	rehash every key; save next first
Open addr.	_probe	% capacity; every slot once
Open addr.	put, get	stop at None, not at a tombstone
Open addr.	delete	leave TOMBSTONE
Open addr.	_resize	drop tombstones; reset both counts

```
pytest tests/test_hashmap.py -v
```

Homework 13 — before Lecture 14

1. **Implement** `dsa/hashmap.py` until all 17 tests pass.
2. **Trace** a `ChainingHashMap(capacity=4)` through `put 5, 9, 13, 2`, then `delete 9`: the chains after each step, and when it resizes.
3. **Trace** an `OpenAddressingHashMap(capacity=8)` through `put 3, 11, 19`, `delete 11`, `get 19`, `put 27`. Show every probe.
4. **Break it.** Replace `TOMBSTONE` with `None` in your `delete`. Which test fails, and exactly why?
5. **Measure** probes against load factor in `notebooks/13-hash-tables.ipynb`.

Summary

Seven things to keep

1. **index = hash(key) % capacity** — computed, not searched.
2. Equal keys **must** hash equal. str hashes change **between runs**; int hashes do not.
3. Collisions are certain and early: $\sim 1.18\sqrt{m}$ keys (birthday paradox).
4. **Chaining**: a linked list per bucket; cost about $1 + \alpha$.
5. **Linear probing**: next free slot; clustering; delete leaves a **tombstone**.
6. Keep α bounded: **double and rehash every key** — amortised $O(1)$.
7. $O(1)$ is the **average** for well-spread keys; the worst case is $O(n)$, and an adversary can force it.

Week 14 — Graphs. The most general structure in the course — stored in the hash map you just built — and the two searches that walk it: breadth-first and depth-first.